

PRESENTATION SCRIPT

Purchase Intent Scorer — e-commerce session scoring

Your speaking notes. Blue boxes are what to say out loud; everything else is backup you only need if asked.

BEFORE YOU START — 60-second checklist

1. Open the Space and let it wake up. Free tier sleeps; a cold start takes ~30 seconds.
2. Open the green GitHub Actions run in a second tab.
3. Score one session before the audience arrives, so the model is warm and the first click is instant.
4. Have the numbers page of this document open on your phone.

The 30-second version (if you only get one minute)

SAY THIS

Only about 15% of e-commerce sessions end in a purchase. Discount everyone and you destroy your margin on the people who were going to buy anyway; discount nobody and you lose the ones you could have won.

We built a model that scores a live browsing session and tells you which of those two groups it is in — so the marketing budget goes to sessions that are winnable but not yet won.

It's deployed on Hugging Face, and every push to GitHub redeploys it automatically after the tests pass.

Number	Value	What it means
Sessions analysed	12,205	after removing 125 duplicate rows
Conversion rate	15.6%	the imbalance that drives every decision
Best model	XGBoost	beat Logistic Regression and Random Forest
ROC-AUC	0.9367	0.5 is a coin flip, 1.0 is perfect
PR-AUC	0.7546	the metric we actually selected on
Prediction speed	10.4 ms	fast enough to run live on a website
Model size	0.47 MB	fits the free hosting tier easily

1. The live demo — click by click

Do the demo early. It buys attention for the technical part.

Step 1 — Score a high-intent session

- Click the preset “High-intent buyer”, then “Score this session”.
- Point at the gauge: it reads 93.6%, well past the 70% line.

SAY THIS

This visitor has viewed 29 product pages and spent about 19 minutes on the site. The model puts them at 93.6% — nearly six times the site average of 15.6%.

The recommendation is deliberately NOT to discount. They are already going to buy; a voucher here is pure lost margin.

Step 2 — Score a bouncer, to show contrast

- Click “Quick bouncer” → “Score this session”. It drops to about 1%.

SAY THIS

Same model, completely different verdict — one page, immediate exit. Spending anything on this session is wasted.

Step 3 — Open the “Why this score” tab (this is the bit people remember)

SAY THIS

The model isn't a black box. This chart breaks the individual prediction into the exact contribution of every field.

For this session, PageValues pushed it up hardest, then the fact that it happened in November. That is a per-prediction explanation, not a general statement about the model.

If someone asks how that works

It's SHAP — a method from game theory that splits a prediction between its features fairly. We use XGBoost's own exact implementation, so no extra library is needed.

Step 4 — The what-if tab

- Leave the field on PageValues and let them watch the curve.

SAY THIS

This holds everything else fixed and sweeps one field. You can see exactly where the model changes its mind — and if the line were flat, that field simply isn't what's driving this decision.

Step 5 — Batch scoring

- Upload a CSV of sessions; it ranks all of them by intent.

SAY THIS

In practice a marketer doesn't score sessions one at a time — they export a day of traffic, upload it, and get every session ranked so they can target the top slice.

Strong move if you have 20 spare seconds

Upload a deliberately wrong file — any unrelated dataset. The app refuses it and reports how many of the 17 required fields it recognised. Say: “We tested it with the wrong data on purpose. It fails loudly instead of returning confident nonsense.” That single line demonstrates engineering judgement better than any metric.

2. The data and what we found

Where it came from

- UCI Machine Learning Repository — Online Shoppers Purchasing Intention (CC BY 4.0).
- 12,330 real sessions, 17 input fields, one yes/no outcome: did it end in a purchase.
- Removed 125 exact duplicate rows → 12,205 used. Split 9,764 train / 2,441 test, stratified.

Four findings worth mentioning

SAY THIS

First, the data is imbalanced — only 15.6% of sessions convert. That single fact drove every later decision.

Second, four columns arrive as integers but aren't numbers at all — Browser, Region, OperatingSystems, TrafficType are ID codes. Browser 13 is not 'greater than' Browser 2, so we one-hot encoded them instead of scaling them.

Third, PageValues is by far the strongest signal — correlation 0.49 with the outcome, and roughly triple the mutual information of anything else.

Fourth, there are no January or April sessions in the data at all, so we simply cannot claim anything about those months.

The “outlier” point examiners like

2,730 PageValues rows flag as statistical outliers — but that is an artefact: the median is 0, so the interquartile range is 0 and *any* non-zero value looks extreme. These are genuine high-value sessions, not errors. We kept them. Deleting them would have deleted the signal.

3. Models and why XGBoost won

Model	PR-AUC	ROC-AUC	Recall	Speed
Logistic Regression	0.6701	0.9157	0.7644	8.2 ms
Random Forest	0.7428	0.9328	0.8115	48.9 ms
XGBoost ← chosen	0.7546	0.9367	0.8377	10.4 ms

SAY THIS

We trained three models and XGBoost won on every ranking metric — and it's nearly five times faster than Random Forest, which matters because this is meant to run live on a website.

XGBoost builds 400 small decision trees where each one corrects the mistakes of the ones before it. That sequential correction is why it edges out Random Forest, which builds its trees independently.

Why PR-AUC and not accuracy — know this one cold

SAY THIS

If we'd used accuracy, a model that predicts 'nobody buys' every single time scores 84.5% — and is completely worthless commercially.

PR-AUC focuses on the rare class we actually care about, so it can't be flattered by the 84% who don't buy. That's why we selected on it.

Tuning actually moved the needle

- Random Forest: 0.7174 → 0.7515 PR-AUC (+0.0341)
- XGBoost: 0.7232 → 0.7540 PR-AUC (+0.0308, in only 20 seconds of search)

RandomizedSearchCV with 5-fold stratified cross-validation, scored on average precision.

The 70% threshold

SAY THIS

The model outputs a probability, but a business needs a yes or no. We swept every cut-off from 5% to 95% on the held-out test set and picked 70%, which maximises F1.

At that setting, out of 2,441 test sessions: 268 real buyers correctly caught, 128 false alarms, 114 buyers missed.

If challenged on the missed buyers

That is a deliberate trade, not an oversight. Catching those 114 means lowering the threshold, which means paying for far more false alarms. The app has a toggle for the profit-maximising cut-off at 75% so you can show both policies live.

4. The two findings that show real rigour

Lead with these. Volunteering a weakness reads as confidence, and both have a good answer.

Finding 1 — The model leans heavily on one feature

SAY THIS

PageValues is about a quarter of the model's decision-making, and Google Analytics computes it partly from transaction revenue — so it's partially derived from the very thing we're predicting.

Rather than hand-wave, we retrained the identical model without it. PR-AUC collapsed from 0.7068 to 0.3437, and recall fell from 0.69 to 0.46.

So we're transparent: the model is strong partly because this feature is strong. In production you'd confirm PageValues is computed from sessions before the current one.

Metric	With PageValues	Without	Change
PR-AUC	0.7068	0.3437	−51%
ROC-AUC	0.9170	0.7546	−18%
Recall	0.6859	0.4581	−33%

Finding 2 — The probabilities are overconfident, on purpose

SAY THIS

We plotted a calibration curve, and the model is overconfident in every single band — on average by about 13 percentage points.

That is not a bug, and we can explain exactly why. Because only 15.6% of sessions convert, we weight the rare buying class about 5.4 times heavier during training to protect recall. That weighting is precisely what inflates the probabilities.

So the scores are reliable for ranking and for thresholding — which is all we use them for — but they shouldn't be read as literal frequencies. Calibrating them would mean giving back some of the recall the weighting bought us.

5. Deployment and the automation

The plain-English version

SAY THIS

Everything — the cleaning, the feature engineering, the encoding and the model — is saved as one single file, 0.47 MB.

That matters more than it sounds. Because the app loads that one file, it is structurally impossible for it to process a live visitor differently from how the training data was processed. That class of bug — train/serve skew — is one of the most common ways real ML systems quietly go wrong.

The CI/CD loop — say it in this order

- I push a change to the main branch on GitHub.
- That automatically starts a GitHub Actions workflow.
- It installs everything, checks the required files exist, loads the model and scores a test session, and runs all 44 tests.
- Only if every check passes does it move to deployment.
- It then copies the app to Hugging Face using an access token stored as an encrypted GitHub secret — the token appears in no file in the repository.
- Hugging Face rebuilds, and the live app updates by itself. No manual upload at any point.

The one-sentence version if time is short

“Push to GitHub → tests run automatically → if they pass it deploys itself to Hugging Face. If they fail, the broken version never reaches the live site.”

We proved it works

SAY THIS

We didn't just build the pipeline — we tested it. We made a visible change, pushed it, and watched the workflow go green and the live app update on its own, with no manual step.

6. Questions you should expect

Short, confident answers. Don't over-explain — stop talking when you've answered.

Q. Why not deep learning / a neural network?

12,205 rows of tabular data. Gradient-boosted trees consistently beat neural networks in that regime, they train in seconds rather than hours, and they give exact per-prediction explanations. A neural net would be slower, harder to explain, and no more accurate.

Q. Isn't 84% accuracy possible by just predicting 'no purchase' every time?

Exactly — 84.5%, and that's precisely why we don't use accuracy. We selected on PR-AUC, which only rewards you for finding the rare class.

Q. What happens if someone uploads the wrong file?

It's rejected, and it tells you how many of the 17 required fields it found. We tested that deliberately with an unrelated dataset. It fails loudly rather than returning confident nonsense.

Q. Is the Hugging Face token safe?

It's stored as an encrypted GitHub Actions secret, never written into any file, and it's masked in the logs. The brief required exactly this and we can show the secret listed without revealing its value.

Q. How would this actually be used by a business?

Their analytics platform scores each live session. Anything in the 55–70% band — winnable but not yet won — triggers an incentive like free shipping. Above that they'd buy anyway, below it the spend is wasted.

Q. What would you do next, with more time?

Three things: confirm PageValues is computed only from prior sessions, calibrate the probabilities so they can be read as real frequencies, and A/B test the incentive to measure actual uplift rather than assuming 10 percentage points.

Q. How do you know it isn't overfitting?

Every number quoted comes from 2,441 sessions the model never saw in training, and tuning used 5-fold cross-validation inside the training set only. All preprocessing lives inside the pipeline, so nothing from the test set can leak into it.

Q. Why is recall only 70% at your threshold?

A deliberate trade. Catching more buyers means lowering the cut-off and paying for many more false alarms. We can switch policies live in the app to show the effect.

The mindset that scores best

The strongest moments here are the two you might be tempted to hide — the PageValues dependence and the calibration finding. Examiners reward measuring your own weaknesses far more than a clean-looking result. Lead with them rather than waiting to be asked.

7. Numbers cheat sheet

The page to glance at on your phone. Everything here came from an actual run.

Data

Item	Value
Raw rows / used	12,330 → 12,205 (125 duplicates removed)
Train / test split	9,764 / 2,441, stratified
Conversion rate	15.6% (imbalance ratio 5.46 : 1)
Missing values	None — zero nulls in the raw file
Months absent	January and April

Chosen model

Item	Value
Algorithm	XGBoost, 400 trees, max depth 6, learning rate 0.01
ROC-AUC / PR-AUC	0.9367 / 0.7546
F1 / Recall / Precision	0.6889 / 0.7016 / 0.6768
Threshold	0.70 (maximises F1); 0.75 maximises modelled value
Confusion (2,441)	TP 268 · FP 128 · FN 114 · TN 1,931
Speed / size	10.4 ms median · 0.47 MB

Honest limits

Item	Value
PageValues dependence	PR-AUC 0.7068 → 0.3437 without it
Calibration	Brier 0.0952, overconfident by ~12.6 pp
Cause	scale_pos_weight 5.4, used to protect recall
Cost assumptions	\$5 incentive, \$120 order, 10pp uplift — assumed, not measured

If the live Space is asleep or broken during the demo

Don't panic and don't apologise twice. Say: “The free tier sleeps when idle — it takes about thirty seconds to wake.” Keep talking through the architecture while it loads. If it genuinely fails, run it locally with `streamlit run app.py` and carry on — the point you're demonstrating is the pipeline, not the hosting.

Links

- GitHub: https://github.com/AP5697/SDAIM_FinalProject
- Live app: https://huggingface.co/spaces/Aishawarya/SDAIM_Final
- Dataset: UCI Online Shoppers Purchasing Intention (ID 468)